

# Practical Practice Sheet - 2

Object Oriented Programming - Simple Java Answers

## 1. Compile-time error - identify the mistake

Mistake: Java does not support multiple inheritance through classes.

A class cannot extend two classes at the same time.

Answer:

```
class Z extends X, Y    // ERROR
```

Correct idea:

```
class Z extends X        // only one superclass
```

## 2. Output of the program

The reference variable is of type A, so a.i is taken from class A.

Output:

10

## 3. Output of the program

When an object of C is created, the instance initialization blocks run from parent to child: A, then B, then C.

Output:

1

2

3

## 4. Output of the program

The object is C. Each class has its own variable s, and super.s moves to the parent class.

Output:

Class B

Class C

## 5. Output of the program

Static blocks run once from the top parent class to the child class when class C is loaded.

Output:

THIRD

SECOND

FIRST

## 6. Output of the program

Constructors execute from the top parent to the child class.

Output:

Class A Constructor

Class B Constructor

Class C Constructor

## 7. Runtime polymorphism - Employee, Developer, Manager

The superclass reference can refer to a Developer or Manager object.

The overridden method is selected at runtime.

```
import java.util.*;

class Employee {
    void displayRole() {
        System.out.println("Role: Employee");
    }
}

class Developer extends Employee {
    @Override
    void displayRole() {
        System.out.println("Role: Developer");
    }
}

class Manager extends Employee {
    @Override
    void displayRole() {
        System.out.println("Role: Manager");
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        Employee e;

        if (n == 1)
            e = new Employee();
        else if (n == 2)
            e = new Developer();
        else
            e = new Manager();

        e.displayRole();
    }
}
```

## 8. Vehicle - Car and Bike

Car and Bike override the same start() method in different ways.

```
import java.util.*;

class Vehicle {
    void start() {
        System.out.println("Vehicle starts.");
    }
}

class Car extends Vehicle {
    @Override
    void start() {
        System.out.println("Car starts with a key.");
    }
}

class Bike extends Vehicle {
    @Override
    void start() {
        System.out.println("Bike starts with a button.");
    }
}
```

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        Vehicle v;

        if (n == 1)
            v = new Car();
        else
            v = new Bike();

        v.start();
    }
}

```

## 9. Student -> CollegeStudent using super

The child constructor calls the parent constructor using super(name).

```

import java.util.*;

class Student {
    String name;

    Student(String name) {
        this.name = name;
    }

    void display() {
        System.out.println("Student Name: " + name);
    }
}

class CollegeStudent extends Student {
    String course;

    CollegeStudent(String name, String course) {
        super(name);
        this.course = course;
    }

    void displayCourse() {
        System.out.println("Course: " + course);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String name = sc.nextLine();
        String course = sc.nextLine();

        CollegeStudent s = new CollegeStudent(name, course);
        s.display();
        s.displayCourse();
    }
}

```

## 10. Payment - Credit Card and UPI

The base class has pay(). Both child classes override it.

```
import java.util.*;

class Payment {
    void pay(int amount) {
        System.out.println("Payment method: Payment");
        System.out.println("Amount: " + amount);
    }
}

class CreditCard extends Payment {
    @Override
    void pay(int amount) {
        System.out.println("Payment method: Credit Card");
        System.out.println("Amount: " + amount);
    }
}

class UPI extends Payment {
    @Override
    void pay(int amount) {
        System.out.println("Payment method: UPI");
        System.out.println("Amount: " + amount);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int choice = sc.nextInt();
        int amount = sc.nextInt();

        Payment p;

        if (choice == 1)
            p = new CreditCard();
        else
            p = new UPI();

        p.pay(amount);
    }
}
```

## 11. Multilevel inheritance - Person, Student, GraduateStudent

One object of GraduateStudent can use methods from all three classes.

```
import java.util.*;

class Person {
    String name;

    Person(String name) {
        this.name = name;
    }

    void displayPerson() {
        System.out.println("Name: " + name);
    }
}

class Student extends Person {
    String studentId;

    Student(String name, String studentId) {
        super(name);
        this.studentId = studentId;
    }
}
```

```

    void displayStudent() {
        System.out.println("Student ID: " + studentId);
    }
}

class GraduateStudent extends Student {
    String researchArea;

    GraduateStudent(String name, String id, String area) {
        super(name, id);
        researchArea = area;
    }

    void displayGraduateStudent() {
        System.out.println("Research Area: " + researchArea);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String name = sc.nextLine();
        String id = sc.nextLine();
        String area = sc.nextLine();

        GraduateStudent g =
            new GraduateStudent(name, id, area);

        g.displayPerson();
        g.displayStudent();
        g.displayGraduateStudent();
    }
}

```

## 12. Constructor order - Grandparent, Parent, Child

When a Child object is created, the superclass constructors run first.

```

class Grandparent {
    Grandparent() {
        System.out.println("Grandparent constructor");
    }
}

class Parent extends Grandparent {
    Parent() {
        System.out.println("Parent constructor");
    }
}

class Child extends Parent {
    Child() {
        System.out.println("Child constructor");
    }
}

public class Main {
    public static void main(String[] args) {
        Child c = new Child();
    }
}

```

Note: Answers are kept simple and follow the questions in the uploaded Practical Practice Sheet - 2.